Documento de Especificaciones Funcionales (FRS)

Proyecto: NextGen Fantasy – LaLiga Edition

1. Visión General del Producto

NextGen Fantasy – LaLiga Edition es una plataforma móvil de gestión deportiva (Fantasy) de nueva generación. El objetivo principal del producto es disrumpir el mercado actual resolviendo problemas endémicos del sector, concretamente el estancamiento de plantillas (inmovilismo de jugadores estrella) y la linealidad de la experiencia de usuario.

La aplicación combina el cálculo estadístico objetivo en tiempo real con una profunda capa de **gestión financiera, teoría de juegos y gamificación**, transformando al usuario de un simple seleccionador a un estratega deportivo y financiero.

---

2. Arquitectura de Usuarios y Sistema de Ligas

El sistema debe soportar un entorno multijugador altamente concurrente, estructurado en diferentes modalidades de competición.

2.1. Tipos de Ligas

- **Ligas Privadas (Peer-to-Peer):** Entornos cerrados autogestionados por los usuarios.
    - *Onboarding dinámico:* Capacidad de invitar a nuevos mánagers mediante la generación de un enlace único (Deep Link) o código QR. Al hacer clic, el usuario descarga la app (si no la tiene) y entra directamente a la liga.
- **Ligas Públicas Oficiales (Globales):** Ligas creadas automáticamente por la plataforma (ej. "Liga NextGen Global", "Liga de Novatos") donde los usuarios solitarios pueden unirse y competir contra otros mánagers de nivel similar mediante un sistema de *matchmaking* básico.

2.2. Castigo Semanal al "Farolillo Rojo" (Novedad)

- **Mecánica:** Al finalizar cada jornada, el usuario que quede en última posición en la clasificación parcial de su liga privada recibirá un "Castigo Fantasy".
- **Ejecución:** El sistema asignará automáticamente un avatar humillante (configurable por el administrador de la liga), un apodo temporal, o una ligera penalización financiera (ej. reducción del 2% de los ingresos de la jornada) para fomentar la competitividad hasta el último puesto y evitar el abandono de la liga.

---

3. Core Funcional: El Motor de Juego

3.1. Sistema de Puntuación Automatizado

- **Fuente de Datos:** Integración con proveedores de estadísticas reales (tipo Opta o DAZN True Stats).
- **Algoritmo de Puntuación:** Cálculo automático y en tiempo real basado en el rendimiento objetivo (goles, asistencias, pases clave, intercepciones, porterías a cero). Se elimina la subjetividad periodística ("picas").

3.2. Gestión de Alineaciones y Roles Especiales

El mánager debe configurar su once inicial antes del pitido inicial del primer partido de la jornada.

- **Formaciones:** Selección de sistemas tácticos estándar (4-4-2, 4-3-3, 3-5-2, etc.).
- **Capitán (De serie):** El mánager designa a un jugador cuyo puntaje final se multiplicará por 2 (x2).
- **Entrenador (De serie):** Permite otorgar bonificaciones específicas a una línea del campo (ej. +10% a la defensa) o realizar ajustes de formación de última hora.
- **Banquillo Activo:** Sistema inteligente de suplencias.
    - *Opción A (Sustitución):* Si un titular juega 0 minutos, el suplente de su misma posición entra automáticamente.
    - *Opción B (Aporte Porcentual):* El usuario puede configurar que el banquillo siempre aporte un porcentaje de sus puntos (ej. 20% de los puntos de los suplentes se suman al total), premiando a los mánagers con plantillas amplias y equilibradas.

3.3. Innovación: Gestión de Alineación Incompleta (Penalización Proporcional)

Para proteger a los usuarios con bajo presupuesto sin premiar la especulación extrema de los líderes:

- **Regla de Negocio:** Si un mánager no logra completar el once inicial (deja huecos vacíos), **no** se invalidará su puntuación ni se restarán puntos fijos (-4 pts).
- **Cálculo:** Se aplicará una **reducción del 20% sobre el total de los puntos obtenidos en la jornada**.
- *Ejemplo de impacto:* Un equipo élite que sume 80 puntos con 10 jugadores perderá 16 puntos (se queda en 64). Un equipo modesto que sume 20 puntos con 10 jugadores perderá 4 puntos (se queda en 16).

---

4. Sistema Financiero y Mercado Disruptivo

Este es el núcleo innovador de NextGen, diseñado para inyectar liquidez y movimiento continuo en el mercado.

4.1. El Sistema de Blindaje ("Impuesto de Cláusula")

Se elimina el bloqueo estático de jugadores estrella mediante un modelo de economía inflacionaria inversa.

- **Definición de Cláusulas:** El usuario es propietario de los jugadores y define libremente su cláusula de rescisión (precio de venta obligatorio) utilizando moneda ficticia del juego.
- **Tasa Semanal de Mantenimiento:** El sistema deducirá automáticamente de la caja del equipo un **1% semanal** del valor total de la cláusula de cada jugador.
    - *Ejemplo:* Blindar a Mbappé con una cláusula de 100M costará 1M por semana.
- **Dinámica de Mercado:** Esto obliga al usuario a balancear el riesgo. Si baja la cláusula para no pagar impuestos, otro mánager puede pagar la cláusula ("Clausulazo") y robarle al jugador. Si la sube, se descapitaliza semana a semana.

4.2. Gestión de Saldo Negativo e Hipotecas (Nueva Mecánica)

A diferencia de otras apps donde estar en negativo impide puntuar, NextGen permite operar en números rojos bajo un sistema de crédito estricto:

- **Penalización por Deuda:** Un usuario puede empezar la jornada en negativo y puntuar, pero el sistema le cobrará un interés de demora acumulativo (ej. 5% de su deuda cada semana que no sanee sus cuentas).
- **Bancarrota e Hipoteca Forzosa:** Si el saldo negativo del usuario **supera el 30% del valor total de su plantilla**, el sistema interviene. El usuario entra en "Estado de Hipoteca" y se le obliga a vender activos al mercado (a precio de coste) hasta sanear la deuda por debajo del umbral crítico.

4.3. Mecánica de Soborno (Sabotaje Financiero)

Una capa de interacción PvP (Jugador contra Jugador) inédita en el mercado:

- **Acción:** El Usuario A quiere que el Usuario B no alinee a su jugador estrella (ej. Vinícius Jr.) en la jornada actual.
- **Oferta:** El Usuario A le envía una oferta de "Soborno" a través de la app, bloqueando una cantidad de su propio saldo (ej. 15 Millones).
- **Resolución:** Si el Usuario B acepta, recibe el dinero inmediatamente, pero el sistema bloquea a Vinícius Jr. en su plantilla y no podrá ser alineado en esa jornada. Si el Usuario B rechaza, el Usuario A recupera su dinero. Esto permite a mánagers con mucho efectivo influir directamente en los partidos de sus rivales.

---

5. Gamificación: El Meta-Juego de Sobres y Tácticas

Para aumentar la retención semanal (Engagement) y la profundidad estratégica, se integra un sistema de cartas coleccionables de un solo uso.

5.1. "Sobres de Táctica" Semanales

- **Distribución:** Cada lunes, todos los usuarios reciben una notificación push para abrir su "Sobre de Táctica" semanal de forma gratuita.
- **Naturaleza de las Cartas:** Las cartas son intransferibles y se almacenan en el inventario del usuario para ser equipadas antes del inicio de cualquier jornada. Solo se puede usar una carta por jornada.
- **Catálogo de Cartas (Ejemplos Funcionales):**
    - *Muro de Nervión:* Otorga +5 puntos extra si los defensas del Sevilla FC en tu alineación mantienen la portería a cero.
    - *Revulsivo de Oro:* El primer jugador que entre desde el banquillo puntúa x2.
    - *Pichichi Express:* Si tu delantero marca en los primeros 15 minutos de la vida real, su gol vale el doble.

---

6. Ecosistema de Apuestas Internas (Dinero Ficticio)

Para proporcionar vías de recuperación de saldo a usuarios descapitalizados y fomentar el consumo de fútbol real.

6.1. Minijuego de Pronósticos

- **Mercados de Cuotas:** Cada jornada, la plataforma publicará "Eventos" con cuotas asociadas (ej. "Iñaki Williams marca: Cuota 2.5", "Girona gana: Cuota 1.8").
- **Mecánica de Apuesta:** Los usuarios pueden invertir una porción limitada de su saldo de mercado (dinero ficticio del juego, evitando así las normativas de "Gambling" real).
- **Resolución:** Si el evento ocurre, el usuario recibe las ganancias al cierre de la jornada, inyectando liquidez fresca en su economía para fichar o subir cláusulas.

---

7. Resumen de Ventaja Competitiva y Posicionamiento

Desde el punto de vista del diseño de producto, **NextGen Fantasy** se posiciona por encima de LaLiga Fantasy Oficial y Biwenger al integrar funcionalidades Premium de manera nativa (Entrenador, Capitán) y crear un bucle de jugabilidad (*Gameplay Loop*) superior.

El modelo penaliza el inmovilismo financiero (mediante el Impuesto del 1%), protege al usuario casual frente a la mala suerte (Penalización porcentual del 20% en huecos) y fomenta interacciones sociales agresivas y divertidas (Sobornos, Castigos al colista). Todo ello empaquetado con mecánicas de monetización enfocadas en la retención pura mediante recompensas aleatorias (Sobres Tácticos), garantizando un MAU (Monthly Active Users) sostenido a lo largo de toda la temporada futbolística.

8. Arquitectura Técnica y Estrategia de Infraestructura Backend (BaaS: Supabase Free Tier)

Para garantizar la viabilidad técnica, el despliegue ágil y la contención total de costes durante las fases de producto mínimo viable (MVP) y validación temprana (2024-2026), **NextGen Fantasy – LaLiga Edition** fundamentará su arquitectura en la capa gratuita de Supabase. Este enfoque exige un diseño de sistemas de alta precisión, micro-optimizaciones a nivel de base de datos y la delegación estratégica de servicios para evitar interrupciones o la degradación del servicio bajo las cuotas restrictivas del plan gratuito.

A continuación, se detalla la especificación técnica exhaustiva para operar dentro de este entorno sin comprometer la escalabilidad del producto.

8.1. Matriz de Cuotas Operativas y Restricciones Arquitectónicas (Plan Gratuito)

El sistema debe estar diseñado para monitorizar y no exceder los siguientes límites estrictos impuestos a nivel de organización. El rebasamiento de métricas críticas (especialmente almacenamiento) paralizará el núcleo transaccional del juego.

Recurso Crítico	**Límite Máximo Permitido**	**Impacto en el Sistema y Medida Arquitectónica a Implementar**
Usuarios Activos Mensuales (MAU)	50.000	Volumen óptimo para MVP. El sistema priorizará la autenticación OAuth para aligerar la carga de gestión de credenciales.
Almacenamiento Base de Datos	**500 MB**	**LÍMITE CRÍTICO.** Superarlo fuerza a PostgreSQL al modo "Solo Lectura", paralizando el mercado y las alineaciones. Exige optimización extrema de esquemas.
Almacenamiento de Archivos (Storage)	1 GB	Límite para recursos binarios. Se implementará una arquitectura híbrida externalizando activos pesados (avatares, escudos).
Transferencia de Salida (Egress)	5 GB (Estándar) + 5 GB (Caché)	Requiere políticas de caché agresivas y compresión

Recurso Crítico	Límite Máximo Permitido	Impacto en el Sistema y Medida Arquitectónica a Implementar
		masiva en el cliente antes de cualquier subida (WebP).
Invocaciones Edge Functions	500.000 / mes	Cuota holgada. Se destinará exclusivamente a lógica de servidor crítica (cierre de jornadas, resolución de sobornos).
Mensajes Realtime (WebSocket)	2 Millones / mes	Se limitará la suscripción en vivo a pantallas de altísima interacción (ej. pujas de última hora en el mercado).
Conexiones Máximas (Directas)	60 (Instancia Nano)	Prohibido el uso de conexiones directas en flujos serverless.
Clientes Máximos (Pooler)	200	Uso obligatorio del servicio Supavisor para la gestión de peticiones concurrentes del frontend.

8.2. Optimización Crítica de PostgreSQL en Entornos Restringidos

La base de datos relacional es el activo más valioso y frágil en el plan gratuito. El diseño del esquema debe priorizar la huella de memoria sobre la conveniencia del desarrollador.

- **Gestión del "Bloat" y Control de Concurrencia Multiversión (MVCC):**
  - Debido a la alta volatilidad del mercado de fichajes de NextGen Fantasy (compras, ventas, fluctuaciones de cláusulas), se generará un alto volumen de "tuplas muertas".
  - **Acción Requerida:** Se programarán rutinas de mantenimiento manuales mediante el comando `VACUUM FULL` en ventanas de baja concurrencia (ej. madrugadas sin partidos) para reclamar el espacio físico al disco, complementando al *autovacuum* nativo del sistema.
- **Eficiencia Estructural y Tipos de Datos:**
  - Se prohíbe el uso de `int8` (BigInt) para identificadores donde `int4` sea suficiente.
  - Los registros históricos (como el desglose de puntuaciones DAZN por jornada) se almacenarán utilizando el tipo `jsonb` con **indexación GIN selectiva**, ahorrando megabytes cruciales frente a esquemas altamente normalizados.

    - **Control de Extensiones:** Solo se instalarán extensiones estrictamente necesarias (como `uuid-ossp` o `pg_stat_statements`), dado que cada extensión activa consume entre 40 MB y 60 MB de la cuota de 500 MB solo en metadatos.
- **Estrategias Avanzadas de Indexación:**
    - **Índices Parciales:** Se implementarán índices con cláusulas `WHERE`. (Ej. Un índice en la tabla de jugadores solo para aquellos con `estado = 'en_mercado'`, reduciendo el tamaño del índice en más de un 70%).
    - **Índices BRIN (Block Range Indexes):** Para tablas de logs temporales o transacciones financieras ordenadas por `created_at`. Producen índices hasta diez veces más pequeños que los tradicionales B-tree.

8.3. Autenticación, Gestión de Identidades y Servicio SMTP Externo

El sistema de identidad (Supabase Auth / GoTrue) presenta una limitación severa en el plan gratuito para el envío de correos electrónicos transaccionales (confirmaciones de registro, recuperación de contraseñas).

- **Restricción:** El servidor SMTP predeterminado de Supabase limita los envíos a 2 o 3 correos por hora, lo que haría colapsar el *onboarding* de cualquier liga.
- **Implementación de SMTP Externo:** Es imperativo sustituir el servidor nativo. La plataforma se integrará con **Resend** (capa gratuita de 3.000 correos/mes) o **SendGrid** (100 correos/día) desde el inicio. Esto se configurará en la sección *Authentication Settings* del dashboard.
- **Prevalencia de OAuth:** Para minimizar el gasto de la cuota SMTP externa, la interfaz de usuario priorizará y destacará visualmente el registro mediante proveedores OAuth (Google, Apple, y GitHub), flujos que no requieren verificación por correo electrónico.

8.4. Lógica Serverless y Optimización de Edge Functions

Las reglas de negocio más complejas (cálculo del impuesto de blindaje del 1%, validación de saldo para sobornos) se ejecutarán en Supabase Edge Functions (entorno Deno).

- **Optimización de Tiempos de Arranque (Cold Starts):** Las funciones aprovecharán las recientes mejoras de infraestructura de Supabase que reducen el arranque en frío en un 97% (aprox. 42 ms).
- **Directrices de Desarrollo en el Edge:**
    - **Dependencias Ligeras:** Se prohíbe importar librerías NPM monolíticas que puedan agotar el límite de tiempo de CPU de 2 segundos.
    - **Almacenamiento Persistente:** Se utilizarán montajes de cubetas S3 para que las funciones accedan rápidamente a configuraciones estáticas de LaLiga (ej. calendarios, escudos) sin realizar costosas peticiones HTTP externas.
    - **Gestión de Secretos:** Las claves de API (ej. para conectarse con proveedores de cuotas de apuestas o DAZN Stats) se inyectarán exclusivamente vía CLI (`supabase secrets set`) y no residirán en el repositorio de código.

8.5. Estrategia Híbrida de Almacenamiento y Mitigación de Egress (Transferencia)

Los límites de 1 GB de Storage y 5 GB de Egress son insuficientes para una app moderna con imágenes de jugadores, escudos de clubes y avatares de usuarios.

- **Arquitectura de Almacenamiento Delegado:**
  - El sistema utilizará **Cloudflare R2** (10 GB gratuitos, $0 de coste por Egress) o **Backblaze B2** como repositorio principal de archivos binarios.
  - La base de datos de Supabase únicamente almacenará el puntero de referencia (la URL pública generada), permitiendo aplicar políticas de Seguridad a Nivel de Fila (RLS) en Postgres sin consumir el valioso almacenamiento nativo.
- **Optimización Extrema de Ancho de Banda (Egress):**
  - **Compresión en Cliente:** Se implementará un módulo en el *frontend* (aplicación móvil) que intercepte cualquier subida de imagen de perfil, la redimensione y la comprima en formato **WebP** antes de la transmisión HTTP (reduciendo archivos de 5MB a ~200KB).
  - **Caché Agresiva:** Todos los activos servidos utilizarán cabeceras estrictas (`Cache-Control: public, max-age=31536000, immutable`) para maximizar la tasa de aciertos del CDN global de Supabase y aprovechar los 5 GB extra de "Egreso en Caché".

8.6. Gestión de Conexiones Concurrentes (Pooler Supavisor)

La aplicación móvil (cliente) y los posibles microservicios *serverless* (ej. Next.js Server Actions) pueden saturar rápidamente las conexiones directas a la base de datos PostgreSQL, provocando errores catastróficos.

- **Prohibición de Conexión Directa:** Queda restringido el uso de la conexión estándar por el puerto 5432 (limitada a 60 conexiones simultáneas y restringida a IPv6 en el plan gratuito).
- **Implementación de Supavisor (Modo Transacción):** Todo cliente de base de datos se conectará obligatoriamente a través del *Connection Pooler* de Supabase por el **puerto 6543**.
  - Se añadirá el parámetro `?pgbouncer=true` en las cadenas de conexión.
  - Esto permite gestionar hasta 200 clientes concurrentes, liberando la conexión inmediatamente después de cada transacción (ideal para el volumen hiper-fragmentado de consultas de una app Fantasy).
  - Garantiza compatibilidad universal al soportar tanto redes IPv4 como IPv6.

8.7. Sistema "Keep-Alive" (Evasión de la Pausa por Inactividad)

Supabase impone una política de pausa automática en proyectos gratuitos que no registren inactividad durante 7 días consecutivos. Las visitas al Dashboard no resetean este contador; se exige ejecución SQL real.

- **Automatización del "Latido" (Heartbeat):**
  - Se configurará un flujo de trabajo programado (Cron Job) utilizando **GitHub Actions**.

    - **Frecuencia y Husos Horarios:** El script se ejecutará dos veces por semana (ej. domingos y jueves a medianoche UTC). Se debe considerar el desfase horario respecto a la hora peninsular española (CET/CEST) para evitar colisiones con los cálculos del Fantasy.
- **Mecánica de Ejecución:**
    - El Action ejecutará un comando `curl` ligero mediante la API REST de Supabase, realizando una operación `SELECT` simple con `limit=1` a una tabla de control (`system_ping`).
    - Se autenticará usando una Service Role Key (protegida como secreto de GitHub), asegurando que el sistema registre la actividad transaccional y mantenga los servidores encendidos indefinidamente, consumiendo menos de 5 minutos mensuales de la cuota de GitHub Actions.

8.8. Flujos de Trabajo (CI/CD), Desarrollo Local y "Exit Strategy"

Para mantener la integridad de los 500 MB y prevenir corrupciones del esquema, se imponen estrictas metodologías de DevOps.

- **Desarrollo Basado en CLI:** Queda prohibida la modificación manual de tablas a través de la interfaz gráfica web ("clic-ops") en el entorno de producción.
- **Ciclo de Vida de Migraciones:**
    1. Los desarrolladores levantarán una réplica exacta de Supabase en contenedores Docker locales (`supabase start`).
    2. Los cambios estructurales se capturarán mediante `supabase db diff`, generando archivos SQL inmutables y versionados.
    3. Tras superar las pruebas, se desplegarán en producción mediante `supabase db push`.
- **Ruta hacia el Auto-alojamiento (Self-Hosting Exit Strategy):**
    - La adopción de Supabase garantiza la ausencia de "Vendor Lock-in". Si NextGen Fantasy alcanza un volumen de usuarios masivo que haga insostenibles económicamente los planes *Pro* gestionados de Supabase, la arquitectura técnica permite la emancipación.
    - Se estipula un protocolo de migración estándar utilizando las herramientas nativas `pg_dump` y `pg_restore` para trasladar íntegramente la base de datos, las políticas RLS y la estructura de autenticación a servidores dedicados propios (VPS tipo Hetzner o AWS EC2) orquestados mediante Docker, garantizando un escalado infinito controlado.

9. Arquitectura Frontend y Estrategia de Ecosistema Cliente (Flutter 2026)

Para garantizar la viabilidad del modelo de negocio a coste inicial cero y asegurar un rendimiento de grado corporativo (60-120 FPS), el desarrollo del cliente multiplataforma de **NextGen Fantasy** se basará exclusivamente en la versión estable 3.41.x de **Flutter**. Este apartado detalla las especificaciones técnicas, decisiones arquitectónicas, dependencias modulares y estrategias de compilación que regirán la construcción de la interfaz de usuario y la lógica de cliente.

9.1. Paradigma Arquitectónico y Lenguaje Base (Dart)

El desarrollo del cliente abandona los enfoques híbridos tradicionales (WebViews o puentes de comunicación asíncronos) en favor del control absoluto a nivel de píxel proporcionado por Flutter y su lenguaje subyacente, **Dart**.

- **Estrategia de Compilación Dual:**
  - **Desarrollo (JIT - Just-In-Time):** Permite la inyección dinámica de código para ciclos de iteración hiper-rápidos.
  - **Producción (AOT - Ahead-Of-Time):** El código se compila directamente en instrucciones máquina nativas (ARM/x64), eliminando la sobrecarga en tiempo de ejecución (runtime overhead) y garantizando arranques instantáneos.
- **Paradigma Declarativo e Inmutabilidad:** Se prohíbe el uso de paradigmas imperativos para la manipulación de la UI. La interfaz debe ser una representación inmutable del estado actual. Ante mutaciones en los datos, el motor realizará una reconciliación algorítmica, reconstruyendo únicamente las variaciones estrictamente necesarias.
- **Ontología Arquitectónica (Los Tres Árboles):**
  1. **Árbol de Widgets (Widget Tree):** Capa pública declarativa (inmutable y efímera).
  2. **Árbol de Elementos (Element Tree):** Instanciación lógica en memoria que enlaza la configuración del widget con la pantalla (mutable).
  3. **Árbol de Objetos de Renderizado (RenderObject Tree):** Capa profunda persistente en memoria encargada del cálculo matemático, restricciones de tamaño (constraints) y pintura final de píxeles.

9.2. Motor de Renderizado, Hilos de Ejecución y Sistemas de Diseño

El ecosistema gráfico de la aplicación debe estar preparado para soportar animaciones complejas, transiciones de datos en tiempo real y componentes vectoriales de alta fidelidad sin bloqueos.

- **Motor Gráfico (Impeller):** Se descarta el uso del antiguo motor Skia. NextGen Fantasy operará forzosamente sobre **Impeller**, optimizado para GPUs modernas (Metal/Vulkan). Esto previene el *jank* (pérdida de fotogramas) mediante la precompilación offline y predictiva de sombreadores (shaders).
- **Fusión de Hilos (The Great Thread Merge):** La arquitectura aprovechará la fusión del hilo de Interfaz de Usuario (UI Thread) y el hilo de Rasterización. Esta directriz técnica reduce la latencia de fotogramas en 1-2 ms y permite el uso de concurrencia nativa (`Isolates`) para cálculos complejos (ej. algoritmos de puntuación en cliente) sin desincronización gráfica.
- **Sistemas de Diseño Integrados:** La aplicación utilizará los componentes de **Material Design 3** y **Cupertino** para mimetizar la física de desplazamiento, tipografías y desenfoques nativos (blur) de Android e iOS respectivamente.

9.3. Requisitos de Infraestructura para el Entorno de Desarrollo

Para garantizar la productividad del equipo de ingeniería y evitar la saturación del hardware durante la compilación AOT y la ejecución de emuladores virtuales, se establecen los siguientes requisitos de estación de trabajo para 2026:

Componente Crítico	**Requisito Mínimo Absoluto**	**Especificación Recomendada (Alta Productividad)**	**Justificación Arquitectónica / Regla de Negocio**
Procesador (CPU)	Intel Core i5 (8ª Gen) / AMD equiv.	Intel Core i7/i9, AMD Ryzen 7+, o Apple Silicon (M-series)	La compilación de Dart y los demonios nativos (Gradle/Xcodebuild) escalan linealmente con múltiples hilos.
Memoria RAM	8 GB	16 GB a 32 GB	Previene la paginación severa a disco (swap) al orquestar simultáneamente IDEs, emuladores AVD y servidores de análisis.
Almacenamiento	512 GB SSD	1 TB NVMe SSD (Gen 4 o superior)	Elude cuellos de botella en lectura/escritura (I/O). El SDK y las carpetas de compilación superan rápidamente los 100 GB.
Aceleración (GPU)	Gráficos integrados básicos	GPU Dedicada (NVIDIA RTX, AMD Radeon) o equivalente Apple	Crítico para acelerar emuladores y soportar el pre-renderizado de Impeller sin asfixiar la CPU principal.
Entorno de Edición	VS Code (Ligero)	Android Studio / IntelliJ (Pesado)	VS Code se impone para equipos al límite de RAM; Android Studio se reserva para

Componente Crítico	Requisito Mínimo Absoluto	Especificación Recomendada (Alta Productividad)	Justificación Arquitectónica / Regla de Negocio
			perfilado de rendimiento profundo.

- **Dependencias de SO:** *Windows* (Git, binarios en Path, Visual Studio C++ toolchain), *macOS* (Xcode-select, Zsh config, Xcode obligatorio para iOS), *Linux* (apt-get dependencias de red y compresión).

9.4. Dinámicas de Compilación y Control del Estado Efímero de Desarrollo

El ciclo de iteración del código se regirá por protocolos estrictos de inyección dinámica para maximizar la velocidad de despliegue en pruebas.

- **Regla de Uso de Hot Reload (Recarga en Caliente):** Obligatorio para ajustes de UI/UX. Inyecta el código modificado en la máquina virtual (JIT) en milisegundos, **preservando el estado de la aplicación** (ej. datos introducidos en un formulario). *Excepción:* Fallará ante modificaciones profundas de jerarquía de clases o tipos enumerados.
- **Regla de Uso de Hot Restart (Reinicio en Caliente):** Obligatorio cuando se altere la arquitectura de estado basal, se inicialicen nuevos controladores lógicos o se inyecten nuevas dependencias. Este comando utiliza el compilador JIT pero **destruye implacablemente el estado en memoria**, reejecutando `main()` y `initState()`.

9.5. Estrategia y Patrones de Gestión de Estado

La orquestación del flujo de datos determinará la escalabilidad del proyecto y su capacidad para no saturar el motor de renderizado con reconstrucciones superfluas. Se categorizan dos dimensiones estrictas:

1. **Estado Ephemeral (Local):** Restringido al ciclo de vida de un widget aislado (ej. índice de un Bottom Navigation Bar o expansión de una tarjeta).
    - *Implementación técnica:* Uso exclusivo del widget nativo `StatefulWidget` y el método `setState()`. Se prohíbe inflar el árbol con librerías externas para efectos visuales simples.
2. **Estado Global (App State):** Datos compartidos jerárquicamente (tokens de autenticación, saldo del usuario, plantillas en tiempo real).
    - *Implementación técnica principal:* **Riverpod (v3.0)**. Se establece como el estándar obligatorio por su capacidad de reconstrucciones hiper-optimizadas mediante escuchas derivadas (`select`), seguridad en tiempo de compilación y recolección de basura automática (`autodispose`). Se usará

extensivamente la familia de modificadores `.family` para dependencias dinámicas.
- *Alternativa (Solo para migraciones legacy complejas):* **BLoC / Cubit**, restringido a módulos que exijan flujos hiper-reactivos puros basados en secuencias de Dart (`Streams`), asumiendo la generación masiva de código repetitivo (boilerplate).

9.6. Integración Modular y Paquetes Core (pub.dev Dependencies)

Para eludir el sobre-desarrollo (reinventar la rueda) manteniendo el binario base optimizado (aprox. 340 KB), se estandariza la inyección de los siguientes módulos corporativos:

Capa Arquitectónica	Paquete Seleccionado	Justificación y Regla de Negocio (Inputs/Outputs)
Comunicaciones HTTP	`dio`	Supera a `http` nativo. Interceptores automatizados para inyección de JWT, validación perimetral y renovación asíncrona de tokens OAuth.
Persistencia Efímera	`shared_preferences`	Almacenamiento rápido en caché de configuraciones simples (ej. modo oscuro, sesión local).
Persistencia NoSQL	`hive`	Serialización documental de alto volumen de lectura/escritura (ej. caché de estadísticas de jugadores) sin relaciones complejas.
Integridad Relacional	`drift`	Mapeo SQL con fiabilidad ACID. Obligatorio si el negocio requiere indexación cruzada verificable en tiempo de compilación.
Enrutamiento (Routing)	`go_router`	Sustituye a Navigator 1.0. Decodifica *Deep Links* nativos de iOS/Android y gestiona URLs dinámicas en web asíncronamente.
Caché Visual Multimedia	`cached_network_image`	Intercepta descargas de imágenes (ej. avatares,

Capa Arquitectónica	Paquete Seleccionado	Justificación y Regla de Negocio (Inputs/Outputs)
		escudos), reteniéndolas en almacenamiento local para evadir llamadas de red letales y repetitivas.
UX Transicional	`skeletonizer`	Sustituye los "spinners" de carga por marcadores espaciales difusos (esqueletos) estáticos perimetrales, mejorando la percepción de velocidad.
Micro-Interacciones	`flutter_animate`	Manejo matemático de transiciones de curvas vectoriales no lineales para efectos hiper-visuales (ej. apertura de sobres Fantasy).

9.7. Matriz Comparativa Definitiva de Infraestructura BaaS (Backend-as-a-Service)

La decisión del backend define la viabilidad financiera del modelo gratuito. El análisis de 2026 expone riesgos asimétricos críticos entre los principales proveedores.

Atributo / Proveedor	Google Firebase (Spark Tier)	Supabase (Open-Source Free Tier)	Appwrite Cloud (Free Tier)
Motor Subyacente	NoSQL Documental (Firestore/RTDB).	Relacional Puro (PostgreSQL).	Híbrido Documental en Docker.
Usuarios (MAU)	50.000 MAU Universales. (Límite OIDC: 50).	50.000 MAU vía GoTrue nativo.	Variable según métricas org.
Almacenamiento BD	1 GiB para registros.	**500 MB exclusivos** (Compartido).	Variable por ancho de banda.
Storage (Archivos)	5 GB para activos estáticos.	1 GB. (Límite max por archivo: 50MB).	1 Bucket. (Límite max por archivo: 50MB).
Tráfico (Egress)	10 GiB Egress DB. RTDB limitado a 100 conexiones.	5 GB globales + Caché perimetral.	Uso equitativo de topología.

Atributo / Proveedor	Google Firebase (Spark Tier)	Supabase (Open-Source Free Tier)	Appwrite Cloud (Free Tier)
Funciones Serverless	**No disponible.** (Exige tarjeta de crédito).	500.000 invocaciones/mes (Motor Deno).	Límite severo: **Solo 2 instancias únicas permitidas.**
Riesgo Estructural	Trampa geométrica de facturación por conteo de Lectura/Escritura.	Parálisis total a los 7 días de inactividad de red. Bloqueo al superar cuota.	Imposibilidad de construir microservicios separados por el límite de 2 funciones.

- **Veredicto Arquitectónico para NextGen Fantasy:** Se adopta **Supabase** como núcleo de datos primario asumiendo la gestión activa del *bloat* relacional, evadiendo así las penalizaciones de facturación NoSQL de Firebase y la asfixia de escalabilidad serverless de Appwrite.

9.8. Estrategias de Despliegue Multiplataforma y Hosting Asíncrono

La aplicación se compilará más allá del espectro móvil, abarcando entornos de escritorio y navegadores web bajo arquitecturas paramétricas estrictas.

- **Adaptación a Escritorio (Desktop OS):** Se exige la implementación del patrón de diseño *Responsive Layout* (mediante `LayoutBuilder`) y el mapeo semántico del ratón (`MouseRegion`). Las inyecciones nativas de barra de menú se operarán vía C FFI.
- **Compilación Web (WebAssembly - Wasm):** Se descarta CanvasKit. En 2026, el despliegue web se compilará en **Wasm**, trasladando el recolector de basura nativo de Dart directamente al motor del navegador. Esto garantiza un rendimiento idéntico a una app nativa, asumiendo un peaje inicial por mayor peso del archivo binario descargado. Soporte garantizado en Chrome v119+ y Firefox v143+.
- **Alojamiento Web Frontend (Hosting Gratuitos 2026):**
    - **Vercel:** Estándar de facto. Otorga 125 GB de tráfico saliente mensual, pero impone un límite rígido de **125.000 peticiones** mensuales.
    - **Appwrite Sites:** Integración recomendada por su exención de límites de transferencia paralela y certificados SSL dinámicos, posicionándose como la alternativa viable en caso de alcanzar los topes de petición de Vercel.
    - **Firebase Hosting:** Limitado severamente a 1 GB o 10 GB. Se clasifica como riesgo crítico a largo plazo debido al peso volumétrico basal de las aplicaciones Wasm.

9.9. Requerimientos No Funcionales (NFRs), UX/UI y Optimización a Futuro

Para mantener un rendimiento óptimo ininterrumpido a 60/120 FPS y preparar la plataforma para futuras innovaciones asíncronas, se definen las siguientes reglas no funcionales obligatorias:

- **NFR-01: Arquitectura Limpia (Clean Architecture).** Obligatoria la separación estricta en capas (Presentación, Dominio, Datos) para evitar el estrangulamiento de código y asegurar escalabilidad plurianual post-2026. Está terminantemente prohibido acoplar la UI directamente a la lógica de base de datos.
- **NFR-02: Uso Fanático del modificador `const`.** Todos los subcomponentes espaciales inmutables deben ser instanciados con el pronombre `const`. Esto erradica la reconstrucción de bucles visuales inalterables, reduciendo dramáticamente el trabajo del Recolector de Basura (Garbage Collector).
- **NFR-03: Renderizado Diferido en Colecciones Masivas (Lazy Rendering).** Las listas, como el mercado de fichajes general, no pueden instanciarse en memoria de forma simultánea. Se exige el uso del patrón *lazy rendering* (`ListView.builder`) y algoritmos reactivos de paginación para no estancar el buffer del GPU.
- **NFR-04: Profiling Físico Obligatorio.** Todo diagnóstico de rendimiento se realizará ejecutando comandos de línea (`flutter run --profile`) exclusivamente sobre hardware real conectado. Los emuladores y simuladores quedan invalidados para auditorías de latencia.
- **Proyección UX/UI (GenUI):** La arquitectura deberá dejar interfaces abstractas (Abstract Classes) preparadas para la futura inyección de Agentes Autónomos y Generación Semántica Inteligente (GenUI), permitiendo que la interfaz transmute su forma automáticamente según el estado del contexto, y se adapte al futuro ecosistema de inyección remota de código en caliente ("efímero") sin pasar por las restricciones de las tiendas (App Store/Google Play).